Open in Colab

AI Web Search (v1)

A minimal RAG-based AI web search demo.

Workflow:

1. User submits a query
2. **Web Search** — Tavily fetches the top 10 results (title, URL, snippet)
3. **LLM Generation** — a single LiteLLM call produces:
   - **Answer**: synthesized response with inline citations
   - **Web Search Results**: all 10 results listed with clickable titles
4. **Display** — markdown output converted to HTML and rendered in Colab

```
# Install required packages
!pip install tavily-python litellm markdown --quiet
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17.0/17.0 MB 83.3 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 278.1/278.1 kB 17.8 MB/s eta 0:00:00
```

Configuration

Set your API keys and choose a model before running.

- **TAVILY_API_KEY**: get one at tavily.com
- **OPENAI_API_KEY**: get one at platform.openai.com
- **MODEL**: any LiteLLM-supported model string (default: `gpt-4o-mini`, cheap and capable)

```
import os

TAVILY_API_KEY = "" # your Tavily API key
OPENAI_API_KEY = "" # your OpenAI API key

os.environ["OPENAI_API_KEY"] = OPENAI_API_KEY  # make key available to LiteLLM

MODEL = "gpt-4o-mini"  # change to e.g. "gpt-4o", "claude-3-haiku-20240307"
TOP_K = 10             # number of web search results to retrieve
```

Step 1: Web Search

The `search_web` function calls the Tavily API and returns the top `k` results. Each result contains a **title**, **URL**, and a **content snippet**.

```
from tavily import TavilyClient

def search_web(query, k=TOP_K):
    """Search the web and return top-k results from Tavily."""
    client = TavilyClient(api_key=TAVILY_API_KEY)
    response = client.search(query, max_results=k)
    # Each result dict: {"title": str, "url": str, "content": str}
    return response["results"]
```

Step 2: Generate Answer with LLM

The `generate_answer` function builds a prompt with all search results and asks the LLM to:

- Answer the query using only the provided results (skip irrelevant ones)
- Add **inline citations** as markdown links `[Title](URL)` near relevant text
- End with a **Web Search Results** section listing every result

```
import litellm
litellm.set_verbose = False  # suppress debug output

def generate_answer(query, results, model=MODEL):
    """Send query + search results to LLM; return a markdown-formatted answer."""
    # Format each result for the prompt, numbered 1..n
    results_text = "\n\n".join(
        f"[{i+1}] Title: {r['title']}\nURL: {r['url']}\nContent: {r['content']}"
        for i, r in enumerate(results)
    )
    n = len(results)

    system_prompt = (
        "You are a helpful research assistant. "
        "Answer the user's query faithfully using ONLY the provided search results. "
        "Do not consider results that are irrelevant to the query. "
        "Add inline citations using the result number as the anchor text: [n](URL) "
        "(e.g. [1](https://example.com)) near supporting text. "
        f"End with a '## Web Search Results' section listing ALL {n} results as:\n"
        "### n. [Title](URL)\nSnippet text"
    )
    user_prompt = f"Question: {query}\n\nSearch Results:\n{results_text}"

    response = litellm.completion(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_prompt},
        ],
    )
    return response.choices[0].message.content  # markdown string
```

```
17:35:01 - LiteLLM:WARNING: common_utils.py:979 - litellm: could not pre-load bedrock-runtime response stream shape -
WARNING:LiteLLM:litellm: could not pre-load bedrock-runtime response stream shape — Bedrock event-stream decoding wil
17:35:02 - LiteLLM:WARNING: common_utils.py:24 - litellm: could not pre-load sagemaker-runtime response stream shape
WARNING:LiteLLM:litellm: could not pre-load sagemaker-runtime response stream shape — SageMaker event-stream decoding
```

Step 3: Display the Answer

Convert the markdown answer to HTML and render it inline in the notebook. This makes citations clickable and the answer easy to read inside Colab.

```
import markdown
from IPython.display import display, HTML

def display_answer(answer_md):
    """Convert markdown answer to HTML and render it in the notebook."""
    html = markdown.markdown(answer_md, extensions=["extra"])
    display(HTML(html))
```

Run the Search

Edit the `query` below and run this cell. The answer will be rendered as HTML.

```
query = "AI Agent Frameworks"  # <- change this

# Step 1: fetch top-10 web results
results = search_web(query)

# Step 2: generate answer with inline citations
answer_md = generate_answer(query, results)

display_answer(answer_md)  # rendered HTML in Colab
```

AI agent frameworks are essential tools for developing intelligent agents that can perform tasks autonomously, make decisions, and interact with users or systems. They streamline the development of agent-based applications by providing key components for reasoning, memory, and tools integration. Notable frameworks include:

1. **LangGraph**: This is tailored for building complex, stateful workflows. It integrates with tools such as LangChain to allow for functionalities like generating motivational quotes or handling automated customer support queries by interfacing with databases and large language models (LLMs) for responses and logging interactions 1.
2. **AutoGen**: A Microsoft-backed framework designed for multi-agent systems where AI agents can collaborate with each other and communicate with humans to automate complex tasks through dynamic conversations 1.
3. **Rasa**: Recognized for its strong governance controls and flexibility, Rasa is ideal for enterprises requiring self-hosting and deterministic behavior in their AI agents. It combines voice and chat functionalities, making it suitable for regulated industries 7.
4. **CrewAI**: Known for robust memory and state management, CrewAI facilitates both isolated tasks and persistent workflows, positioning itself well for applications that entail complex task execution 6.
5. **OpenAI SDK and Google ADK**: These are foundational model labs' in-house frameworks that serve as control layers for managing operations, interactions, and state changes in AI agent processes 2.

In addition to these, there are several other frameworks like Semantic Kernel, Smolagents, and Pydantic AI that cater to various application needs in the realm of AI development, emphasizing the diversity of options available for developers looking to create effective AI agent systems 4 5.

These frameworks collectively offer a robust infrastructure for developers, enabling the creation of sophisticated AI agents that can work autonomously in various operational domains.

Web Search Results

1. AI Agent Frameworks - GeeksforGeeks

AI agent frameworks are tools and platforms that help developers build intelligent agents capable of performing tasks autonomously.

2. Agent Frameworks - Arize AI

AI agent frameworks have matured, with foundational model labs having their own in-house agentic frameworks like OpenAI SDK and Google ADK.

3. AI Agent Frameworks: Choosing the Right Foundation for Your Business

Explores foundational elements, components, and characteristics important in selecting an appropriate agent framework.

4. Comparing Open-Source AI Agent Frameworks - Langfuse

In-depth comparison of leading open-source AI agent frameworks, including features and use cases.

5. Python AI Agent Frameworks Compared: LangGraph vs CrewAI vs ...

A guide comparing multiple Python-based AI agent frameworks to assist developers in choosing the right one.

6. Complete guide to agentic AI frameworks: Comparison and ... - Moxo

Frameworks streamline tasks with capabilities in prompt orchestration, tool integration, and multi-agent coordination.

7. 8 Best AI Agent Frameworks for Enterprise in 2026 | Rasa Blog

An overview of the best AI agent frameworks, evaluating them on deployment flexibility and production readiness.

8. What's the best framework for production-grade AI agents right now?

Discussion on solid choices for building production-grade AI agents.

9. Agent Framework: Building Blocks for the Next Generation of AI Agents